DOCUMENTO OFICIAL DE INGENIERÍA TDD

Catálogo Exhaustivo y Detallado de Pruebas

Especificación Minuciosa: 25 Pruebas Implementadas + 15 Pruebas Planificadas (AAA Completo)

EQUIPO DE DESARROLLO	TOTAL DE CASOS	ESTADO IMPLEMENTADAS	ESTADO PLANIFICADAS
Andrés & Cristian Beltrán	40 Pruebas Exhaustivas	25 Pruebas en Verde (OK)	15 Pruebas (Auth, IAM, E2E)

📖 **¿Cómo está detallada cada prueba en este documento?**

Cada una de las 40 pruebas cuenta con su ficha técnica dividida en las 3 fases universales de pruebas:

- **Preparar (Arrange):** Describe exactamente qué datos, modelos y entidades se colocan en la base de datos de pruebas (SQLite en memoria).
- **Actuar (Act):** Describe la función, método de servicio o petición HTTP (GET, POST, PATCH) que se dispara.
- **Verificar (Assert):** Describe las aserciones obligatorias (códigos de estado HTTP, cantidad exacta de elementos, igualdad de valores y tipos de datos).

1. Diagrama Relacional de Referencia (Base de Datos medicent_v3) 🖼️

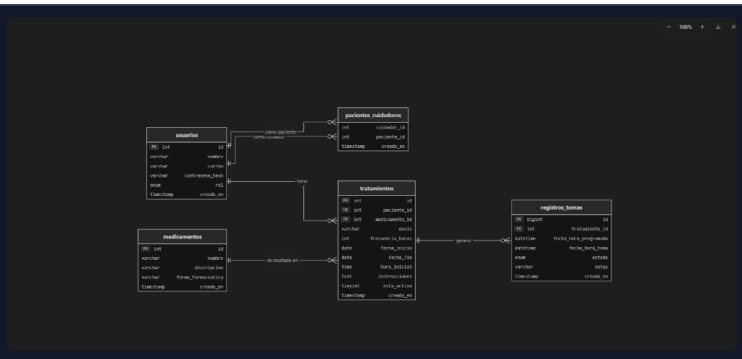


Figura 1: Entidades del sistema (usuarios, pacientes_cuidadores, medicamentos, tratamientos, registros_tomas).

2. Resumen de la Matriz de Pruebas por Módulos

Nivel	Módulo / Dominio	Implementadas	Planificadas	Total	Archivos Asociados
Unitarias (Nivel 1)	Matemática de tomas, instanciaciones puras y validaciones	1 caso	5 casos	6 casos	`tests/unit/test_calculo_tomas.py`
Integración (Nivel 2)	Catálogo clínico, motor de tratamientos, seguimiento de tomas y endpoints REST	24 casos	5 casos	29 casos	`tests/integration/test_*.py`
End-to-End (Nivel 3)	Recorridos completos de API (Registro, Login, Prescripción y Auditoría)	0 casos	5 casos	5 casos	`tests/e2e/test_flujo_clinico_e2e.py`
TOTAL GENERAL		**25 IMPLEMENTADAS (OK)**	**15 PLANIFICADAS**	**40 CASOS**	Suite completa de pruebas

3. Catálogo Detallado de Pruebas Unitarias (Nivel 1 — Funciones Aisladas) 🧪

Operan en memoria pura sin tocar base de datos de disco ni requerir servidor web activo.

U1 `test_algoritmo_calculo_tomas_intervalo_regular` — UNITARIA · IMPLEMENTADA · `tests/unit/test_calculo_tomas.py`

Objetivo: Validar la matemática de calendarización: para un tratamiento de 2 días (48 horas) cada 8 horas, debe generar exactamente 6 timestamps con intervalos idénticos de 8 horas.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Fecha inicio: 2026-11-01, Fecha fin: 2026-11-02, Hora inicial: 08:00:00, Frecuencia: 8 horas.	Invocar `calcular_horarios_tomas(...)` con los parámetros definidos.	`len(horarios) == 6`, `horarios[0] == 2026-11-01 08:00:00`, y cada diferencia temporal es exactamente de 8 horas.

U2 `test_modelo_usuario_instanciacion_y_atributos` — UNITARIA · PLANIFICADA · `tests/unit/test_modelos_unitarios.py`

Objetivo: Validar que la clase `Usuario` asigne sus propiedades en memoria y que su método de serialización oculte el hash de contraseña.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Atributos en memoria: nombre='Carlos', correo='carlos@medicent.com', contrasena_hash='hash123', rol='cuidador'.	Instanciar objeto `Usuario(...)` y llamar a su método de serialización `to_dict()`.	Los atributos coinciden con los provistos y la clave `contrasena_hash` no existe en el diccionario retornado.

U3 `test_modelo_medicamento_instanciacion` — UNITARIA · PLANIFICADA · `tests/unit/test_modelos_unitarios.py`

Objetivo: Validar la instanciación en memoria del modelo `Medicamento` con sus atributos farmacológicos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Nombre='Loratadina', descripción='Antihistamínico', forma='Comprimido 10mg'.	Instanciar `Medicamento(...)` y ejecutar `to_dict()`.	Los campos coinciden y el diccionario contiene las claves 'nombre', 'descripcion' y 'forma_farmaceutica'.

U4 `test_modelo_tratamiento_instanciacion_y_defaults` — UNITARIA · PLANIFICADA · `tests/unit/test_modelos_unitarios.py`

Objetivo: Comprobar que una instancia de `Tratamiento` mantenga activo su estado booleano por defecto.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Parámetros válidos de prescripción médica en memoria.	Instanciar `Tratamiento(...)` sin especificar `esta_activo`.	El campo `esta_activo` se inicializa como `True` y las fechas son coherentes.

U5 `test_modelo_registro_toma_estado_inicial` — UNITARIA · PLANIFICADA · `tests/unit/test_modelos_unitarios.py`

Objetivo: Confirmar que una instancia de `RegistroToma` inicie con estado 'pendiente' y fecha_hora_toma nula.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Timestamp programado y tratamiento_id válido en memoria.	Instanciar `RegistroToma(tratamiento_id=1, fecha_hora_programada=...)`.	`estado == 'pendiente'` y `fecha_hora_toma is None`.

U6 `test_validacion_algoritmo_tomas_frecuencia_invalida` — UNITARIA · PLANIFICADA · `tests/unit/test_calculo_tomas.py`

Objetivo: Garantizar que el motor matemático rechace frecuencias menores o iguales a cero para evitar bucles infinitos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Rango de fechas válido con frecuencia = 0 o frecuencia = -4.	Invocar función de cálculo de horarios dentro de un bloque de aserción de excepción.	Se lanza `ValueError` con mensaje descriptivo de frecuencia inválida.

4. Catálogo Detallado de Pruebas de Integración (Nivel 2 — Servicios, BD y Rutas) 🧩

Verifican la colaboración entre modelos ORM, base de datos SQLite en memoria, servicios de lógica de negocio y controladores HTTP.

I1 `test_creacion_modelo_medicamento`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento.py`

Objetivo: Persistir un medicamento en la base de datos de pruebas y verificar que se genere su ID autoincremental primario.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Medicamento('Paracetamol', 'Analgésico y antipirético', 'Comprimido 500mg').	`db.session.add(med); db.session.commit()`.	Consultar por nombre en BD: `id is not None`, forma farmacéutica exacta y serialización coincide.

I2 `test_crear_medicamento_exitoso`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_service.py`

Objetivo: Validar que `medicamento_service.crear_medicamento()` guarde correctamente en BD un fármaco válido.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Nombre: 'Ibuprofeno', Descripción: 'Antiinflamatorio', Forma: 'Cápsula 400mg'.	Ejecutar `crear_medicamento(...)`.	El objeto retornado posee `id` válido, nombre persistido y forma farmacéutica almacenada.

I3 `test_crear_medicamento_sin_nombre_lanza_error`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_service.py`

Objetivo: Asegurar que el servicio valide la presencia obligatoria del nombre del fármaco y rechace cadenas vacías.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Nombre vacío `""` o compuesto solo por espacios en blanco.	Invocar `crear_medicamento(nombre="")` dentro de `assertRaises`.	Lanza `ValueError` conteniendo la palabra "nombre" en el mensaje de error.

I4 `test_obtener_medicamentos_retorna_lista`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_service.py`

Objetivo: Crear dos medicamentos en BD y comprobar que el servicio retorne el listado completo conteniendo ambos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Registrar 'Amoxicilina' y 'Omeprazol' en la base de datos de pruebas.	Llamar a `obtener_medicamentos()`.	La lista tiene longitud 2 y ambos nombres están contenidos en los resultados.

I5 `test_obtener_medicamento_por_id_existente_e_inexistente`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_service.py`

Objetivo: Validar consulta por ID: retorna la instancia si el registro existe o `None` si el ID no está registrado.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Fármaco 'Loratadina' guardado con ID generado; ID inexistente 9999.	Llamar a `obtener_medicamento_por_id(...)` con ambos identificadores.	Retorna el objeto válido con el ID real y `None` con el ID 9999.

I6 `test_crear_medicamento_endpoint_exitoso`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_routes.py`

Objetivo: Petición HTTP POST a `/api/medicamentos` debe responder código 201 Created y JSON del fármaco creado.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Payload JSON con nombre, descripción y forma farmacéutica.	`self.client.post("/api/medicamentos", json=payload)`.	Código HTTP 201, mensaje de éxito en JSON y objeto medicamento con ID asignado.

I7 `test_crear_medicamento_sin_nombre_retorna_400`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_routes.py`

Objetivo: Petición POST a `/api/medicamentos` sin el campo obligatorio "nombre" debe responder 400 Bad Request.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Payload omitiendo la clave "nombre".	Enviar POST a la ruta del endpoint.	Código HTTP 400 y mensaje de error descriptivo en la respuesta JSON.

I8 `test_listar_medicamentos_endpoint`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_medicamento_routes.py`

Objetivo: Petición HTTP GET a `/api/medicamentos` debe responder 200 OK y la lista de medicamentos en formato JSON.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Registrar 2 medicamentos previamente en la base de datos de pruebas.	`self.client.get("/api/medicamentos")`.	Código HTTP 200, array JSON con 2 elementos y nombres coincidentes.

I9 `test_creacion_modelo_tratamiento`

INTEGRACIÓN | IMPLEMENTADA | `tests/integration/test_tratamiento.py`

Objetivo: Comprobar la integridad de claves foráneas entre `Tratamiento`, `Usuario` y `Medicamento`.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Paciente 'Juan Perez' y Medicamento 'Ibuprofeno' creados en SQLite.	Crear y persistir Tratamiento vinculando ambos identificadores.	Tratamiento guardado con ID; relaciones `guardado.paciente.nombre` y `guardado.medicamento.nombre` resuelven correctamente.

I10 `test_crear_tratamiento_exitoso_genera_tomas` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_service.py`

Objetivo: Al invocar `tratamiento_service.crear_tratamiento()`, deben persistirse automáticamente en cascada todas sus tomas en BD.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Prescripción de 2 días (48h) con frecuencia de 8 horas.	Ejecutar `crear_tratamiento(...)`.	Tratamiento con ID primario, `len(tratamiento.tomas) == 6`, y primera toma en estado 'pendiente'.

I11 `test_crear_tratamiento_paciente_inexistente_lanza_error` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_service.py`

Objetivo: Rechazar la creación de tratamiento si el `paciente_id` no existe en la tabla de usuarios.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Medicamento válido y `paciente_id = 9999`.	Llamar a `crear_tratamiento(...)` dentro de `assertRaises(ValueError)`.	Se lanza excepción indicando que el paciente no existe.

I12 `test_crear_tratamiento_medicamento_inexistente_lanza_error` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_service.py`

Objetivo: Rechazar la creación de tratamiento si el `medicamento_id` no existe en el catálogo farmacológico.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Paciente válido y `medicamento_id = 9999`.	Llamar a `crear_tratamiento(...)`.	Lanza `ValueError` conteniendo la palabra "medicamento".

I13 `test_crear_tratamiento_fechas_o_frecuencia_invalidas` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_service.py`

Objetivo: Validar que `fecha_fin` no sea anterior a `fecha_inicio` y que la frecuencia en horas sea mayor a cero.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Fechas invertidas (inicio: 5 nov, fin: 1 nov) o frecuencia = 0.	Ejecutar servicio creador en ambos casos inválidos.	En ambos casos se arroja `ValueError` bloqueando la inserción.

I14 `test_obtener_tratamientos_por_paciente` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_service.py`

Objetivo: Retornar únicamente los tratamientos pertenecientes al paciente consultado y lista vacía para pacientes sin tratamientos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Crear tratamiento para Paciente A; ID 8888 sin registros.	`obtener_tratamientos_por_paciente(paciente_id)`.	Longitud 1 para el paciente registrado y longitud 0 para el paciente sin datos.

I15 `test_crear_tratamiento_endpoint_exitoso` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_routes.py`

Objetivo: POST a `/api/tratamientos` debe responder 201 Created con tratamiento y conteo de tomas autogeneradas.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Payload JSON completo (paciente_id, medicamento_id, fechas, dosis, frecuencia 8h).	`self.client.post("/api/tratamientos", json=payload)`.	Código HTTP 201, datos del tratamiento y `total_tomas_generadas == 6`.

I16 `test_crear_tratamiento_datos_faltantes_retorna_400` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_routes.py`

Objetivo: POST a `/api/tratamientos` con campos mandatorios omitidos debe responder código 400 Bad Request.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Payload JSON conteniendo únicamente `paciente_id`.	Enviar POST al endpoint.	Código HTTP 400 y mensaje de error en la respuesta JSON.

I17 `test_listar_tratamientos_por_paciente_endpoint` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_tratamiento_routes.py`

Objetivo: GET a `/api/tratamientos/paciente/<id>` retorna 200 OK y la lista de tratamientos de la paciente.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Crear tratamiento vía POST para la paciente Laura.	`self.client.get(f"/api/tratamientos/paciente/")`.	Código HTTP 200, array JSON con longitud 1 y dosis correspondiente.

I18 `test_creacion_modelo_registro_toma` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_registro_toma.py`

Objetivo: Validar la persistencia de un `RegistroToma` vinculado a su tratamiento padre con estado 'pendiente'.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Tratamiento médico persistido en base de datos.	Crear y guardar RegistroToma con fecha/hora programada.	`id is not None`, `estado == 'pendiente'`, y relación `toma.tratamiento.dosis` resuelve bien.

I19 `test_tomas_generadas_estado_inicial_pendiente` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_generacion_tomas.py`

Objetivo: Confirmar que el 100% de las tomas autogeneradas por `generar_tomas_tratamiento()` nazcan en estado 'pendiente'.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Tratamiento de 2 días cada 8h en BD.	Invocar `generar_tomas_tratamiento(tratamiento.id)`.	Exactamente 6 tomas, todas con `estado == 'pendiente'` y `fecha_hora_toma is None`.

I20 `test_transicion_estado_toma_valida` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_marcar_toma.py`

Objetivo: El servicio `toma_service.marcar_toma()` debe actualizar el estado a 'tomada' y asignar timestamp real de toma.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
RegistroToma en estado 'pendiente' en base de datos.	`marcar_toma(toma_id=toma.id, nuevo_estado="tomada")`.	`estado == 'tomada'` y `fecha_hora_toma is not None` tanto en retorno como en BD.

I21 `test_marcar_toma_endpoint_exitoso` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_toma_routes.py`

Objetivo: PATCH a `/api/tomas/<id>` con `nuevo_estado='tomada'` debe responder 200 OK y actualizar la base de datos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Toma pendiente generada para el paciente.	`self.client.patch(f"/api/tomas/", json={"nuevo_estado": "tomada"})`.	Código HTTP 200 y JSON confirmando estado 'tomada' y timestamp asignado.

I22 `test_marcar_toma_estado_invalido_retorna_400` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_toma_routes.py`

Objetivo: PATCH a `/api/tomas/<id>` con estado inventado ('inventado') debe responder 400 Bad Request.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Toma existente; payload con `nuevo_estado="inventado"`.	Enviar PATCH al endpoint.	Código HTTP 400 y mensaje de error en la respuesta JSON.

I23 `test_marcar_toma_inexistente_retorna_404` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_toma_routes.py`

Objetivo: PATCH a `/api/tomas/99999` (toma inexistente) debe responder código 404 Not Found.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
ID inexistente 99999.	`self.client.patch("/api/tomas/99999", json={"nuevo_estado": "tomada"})`.	Código HTTP 404 y mensaje de error en JSON.

I24 `test_listar_tomas_por_paciente_endpoint` INTEGRACIÓN IMPLEMENTADA `tests/integration/test_toma_routes.py`

Objetivo: GET a `/api/tomas/paciente/<id>` retorna 200 OK y el listado de 6 tomas programadas del paciente.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Paciente con tratamiento activo de 6 tomas creadas.	`self.client.get(f"/api/tomas/paciente/")`.	Código HTTP 200, array JSON con 6 tomas y primera toma en estado 'pendiente'.

I25 `test_registrar_usuario_con_hashing_bcrypt` INTEGRACIÓN PLANIFICADA `tests/integration/test_auth_service.py`

Objetivo: `auth_service.registrar_usuario()` debe hashear la contraseña con Bcrypt y persistir el usuario en BD.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Nombre, correo único y contraseña en texto plano 'SuperSecreta2026!'.	Invocar `registrar_usuario(...)`.	`usuario.id is not None`, el hash no es texto plano y valida `True` con Bcrypt.

I26 `test_registro_usuario_correo_duplicado_rechazado` INTEGRACIÓN PLANIFICADA `tests/integration/test_auth_service.py`

Objetivo: Impedir que se registre un segundo usuario si el correo ya existe en la base de datos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Usuario previamente guardado con correo 'mismo@medicent.com'.	Intentar registrar otro usuario con idéntico correo electrónico.	El servicio retorna `None` bloqueando la duplicidad.

I27 `test_iniciar_sesion_exitoso_y_emision_token_jwt` INTEGRACIÓN PLANIFICADA `tests/integration/test_auth_service.py`

Objetivo: `auth_service.iniciar_sesion()` debe validar contraseña y emitir Token JWT firmado con claims de rol.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Usuario 'Carlos Cuidador' registrado con contraseña 'ClaveValida123'.	`iniciar_sesion("carlos.c@medicent.com", "ClaveValida123")`.	Retorna diccionario con clave 'token', rol 'cuidador' y nombre del usuario.

I28 `test_iniciar_sesion_clave_incorrecta_rechazado` INTEGRACIÓN PLANIFICADA `tests/integration/test_auth_service.py`

Objetivo: Comprobar que si la contraseña no coincide con el hash, el servicio retorne `None` y no emita token.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Usuario registrado con contraseña 'Correcta123'.	`iniciar_sesion("carlos@medicent.com", "Errada999")`.	Retorna `None` sin emitir credenciales.

I29 `test_asignar_paciente_a_cuidador_tabla_asociativa` INTEGRACIÓN PLANIFICADA `tests/integration/test_auth_service.py`

Objetivo: `asignar_paciente()` debe persistir la fila en `pacientes_cuidadores` y rechazar asignaciones duplicadas.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
1 cuidador y 1 paciente en BD.	Invocar `asignar_paciente(cuidador.id, paciente.id)` dos veces consecutivas.	La primera llamada retorna el objeto asociado; la segunda llamada retorna `None`.

5. Catálogo Detallado de Pruebas End-to-End (Nivel 3 — Flujos Completos de la API) 🌐

Simulan el comportamiento de clientes HTTP interactuando a través de los Blueprints registrados en la aplicación.

E1 `test_e2e_healthcheck_servidor_y_base_de_datos` — END-TO-END — PLANIFICADA — `tests/e2e/test_flujo_clinico_e2e.py`

Objetivo: Peticiones GET a `/api/estado` y `/api/estado-db` confirman servidor activo y conexión a base de datos.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Instancia de la aplicación levantada con `self.client`.	GET a `/api/estado` y GET a `/api/estado-db`.	Status 200 en ambos endpoints; respuestas JSON confirmando 'activo' y 'conectada'.

E2 `test_e2e_flujo_registro_login_y_obtencion_jwt` — END-TO-END — PLANIFICADA — `tests/e2e/test_flujo_clinico_e2e.py`

Objetivo: Ciclo completo de usuario: registrar una cuenta vía HTTP POST y autenticarse para recibir el Token JWT.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Payload con nombre, correo nuevo y contraseña.	1) POST a `/api/auth/registro`. 2) POST a `/api/auth/login`.	Registro responde 201 Created; Login responde 200 OK y entrega token JWT utilizable.

E3 `test_e2e_login_invalido_rechazado_con_401` — END-TO-END — PLANIFICADA — `tests/e2e/test_flujo_clinico_e2e.py`

Objetivo: Validar que una petición de inicio de sesión con clave errada sea rechazada con código 401 Unauthorized.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Usuario registrado en BD; payload con contraseña incorrecta.	`self.client.post("/api/auth/login", json=payload_invalido)`.	Código HTTP 401 Unauthorized y ningún token emitido en la respuesta.

E4 `test_e2e_ciclo_completo_prescripcion_y_consumo_tomas` — END-TO-END — PLANIFICADA — `tests/e2e/test_flujo_clinico_e2e.py`

Objetivo: Flujo maestro de la API: prescribir tratamiento (3 días cada 12h = 6 tomas), consultar calendario por GET y marcar primera toma como tomada por PATCH.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Paciente y medicamento existentes en base de datos.	1) POST `/api/tratamientos`. 2) GET `/api/tomas/paciente/<id>`. 3) PATCH `/api/tomas/<toma_id>`.	Tratamiento 201 con `total_tomas_generadas == 6`; GET retorna 6 tomas en 'pendiente'; PATCH retorna 200 y el calendario refleja 1 'tomada' y 5 'pendientes'.

E5 `test_e2e_auditoria_de_adherencia_paciente` — END-TO-END — PLANIFICADA — `tests/e2e/test_flujo_clinico_e2e.py`

Objetivo: Comprobar la trazabilidad y auditoría clínica: un cuidador consulta las tomas y audita las fechas reales de consumo y notas registradas.

PREPARAR (ARRANGE):	ACTUAR (ACT):	VERIFICAR (ASSERT):
Paciente con tomas en estados mixtos ('tomada', 'omitida', 'retrasada').	GET a `/api/tomas/paciente/<id>`.	Status 200 y cada elemento del reporte contiene `fecha_hora_programada`, `fecha_hora_toma` y `notas`.

6. Comando Oficial de Ejecución de Pruebas

Para ejecutar las pruebas implementadas con su detalle en consola:

```
py -m unittest discover -v -s tests -p "test_*.py"
```